Introduction to Graphs

Prof. Rohit Pal Singh

Department of Computer Science

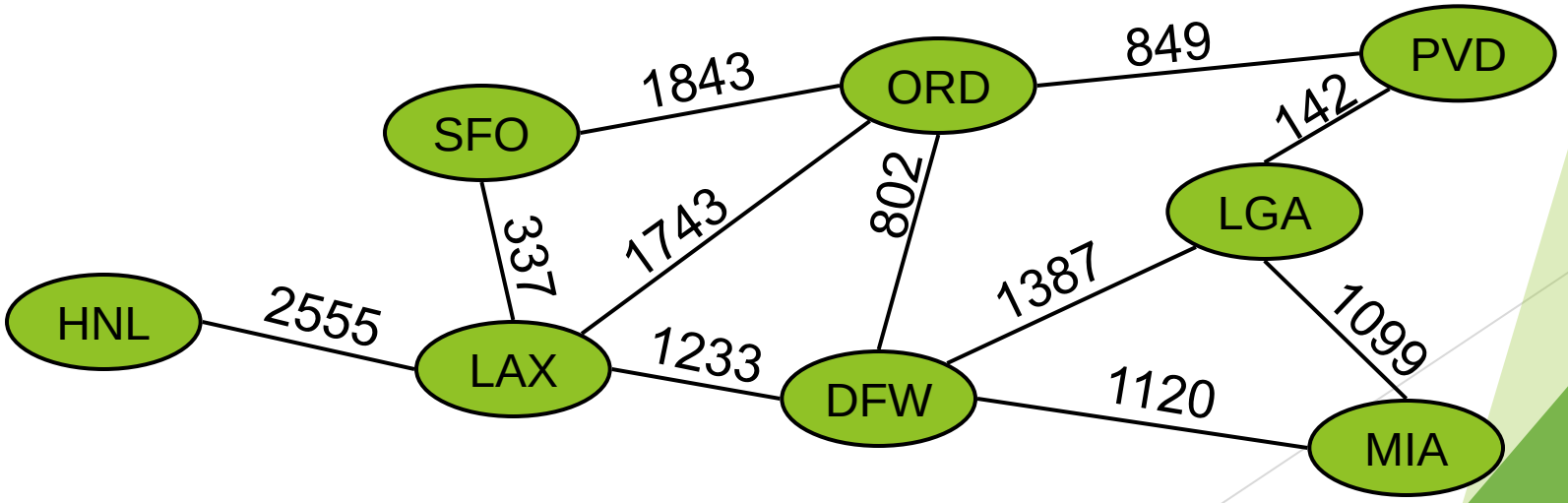
University of Wisconsin, Milwaukee

November 12, 2024

Readings: Chapter 6.1, 6.2

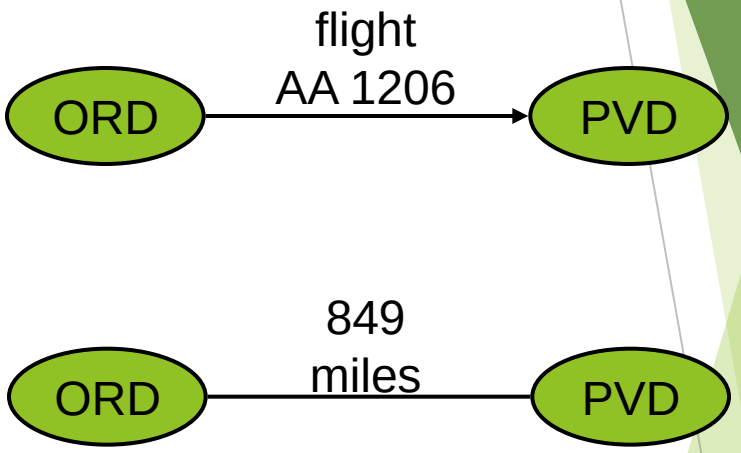
Graph

- A graph is a pair (V, E), where
  - V is a set of nodes, called vertices
  - E is a collection of pairs of vertices, called edges
  -
- Example:
  - A vertex represents an airport and stores the three-letter airport code
  - An edge represents a flight route between two airports and stores the mileage of the route



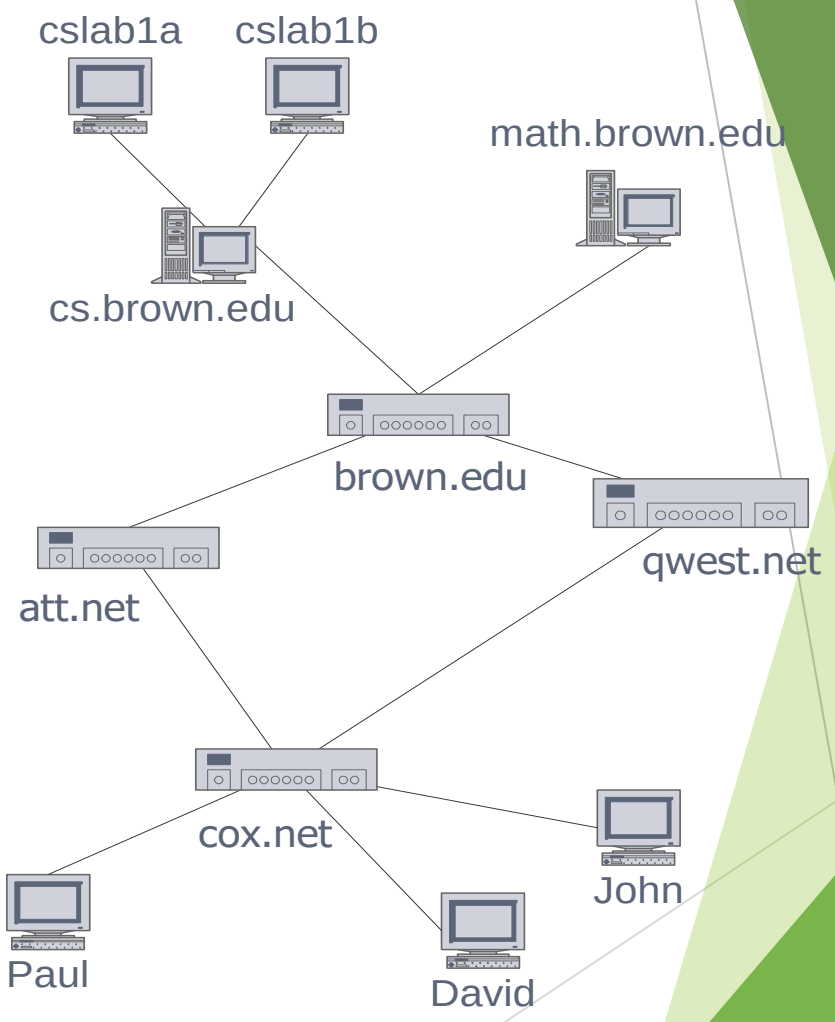
Edge Types

- Directed edge
  - ordered pair of vertices (u,v)
  - first vertex u is the origin
  - second vertex v is the destination
  - e.g., a flight
- Undirected edge
  - unordered pair of vertices (u,v)
  - e.g., a flight route
- Directed graph
  - all the edges are directed
  - e.g., flight network
- Undirected graph
  - all the edges are undirected
  - e.g., route network
- Mixed graph



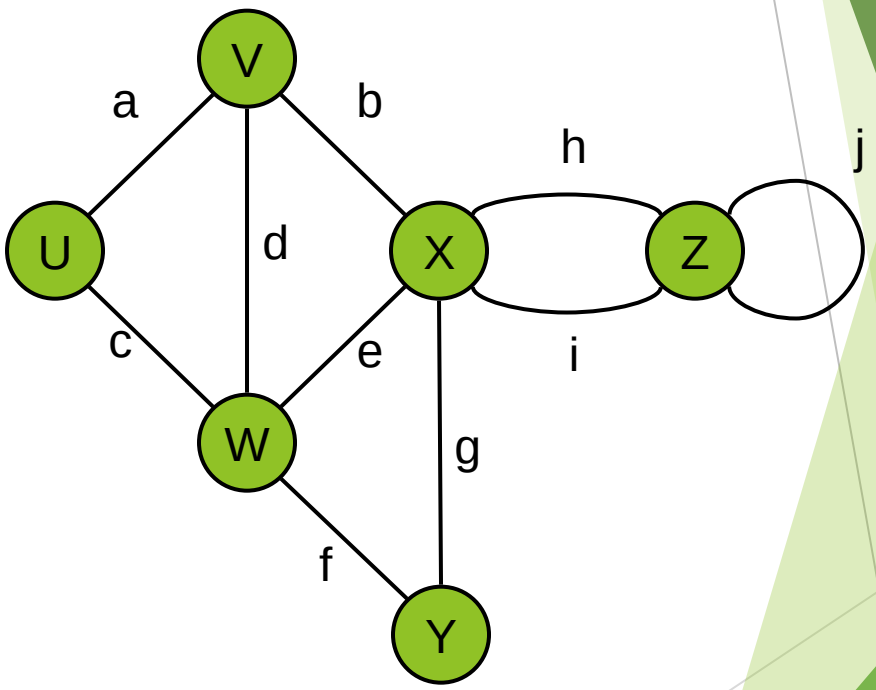
Applications

- Electronic circuits
  - Printed circuit board
  - Integrated circuit
- Transportation networks
  - Highway network
  - Flight network
- Computer networks
  - Local area network
  - Internet
  - Web
- Databases
  - Entity-relationship diagram



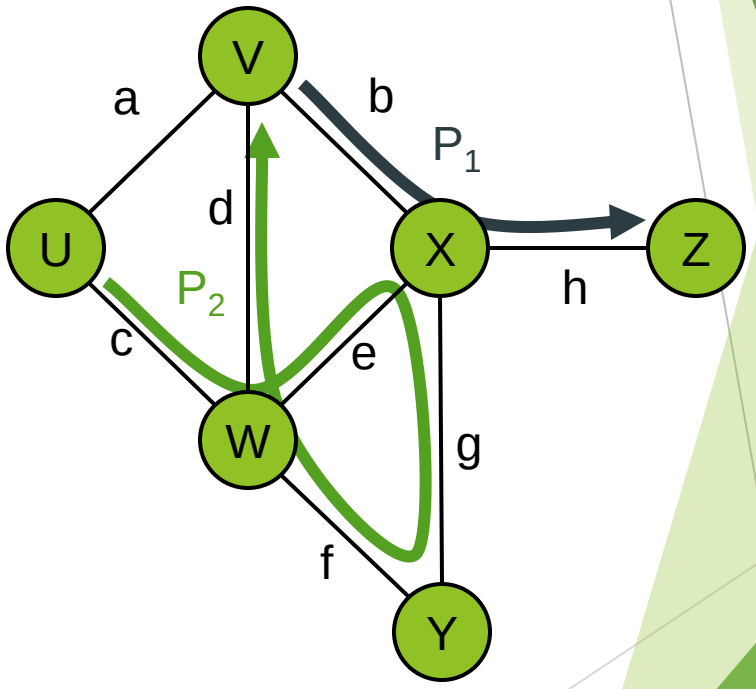
Terminology

- End vertices (or end points) of an edge
  - U and V are the endpoints of a
- Edges incident on a vertex
  - a, d, and b are incident on V
- Adjacent vertices
  - U and V are adjacent
- Degree of a vertex
  - X has degree 5
- Parallel edges
  - h and i are parallel edges
- Self-loop
  - j is a self-loop
- Simple graph
  - No parallel edges or self-loops



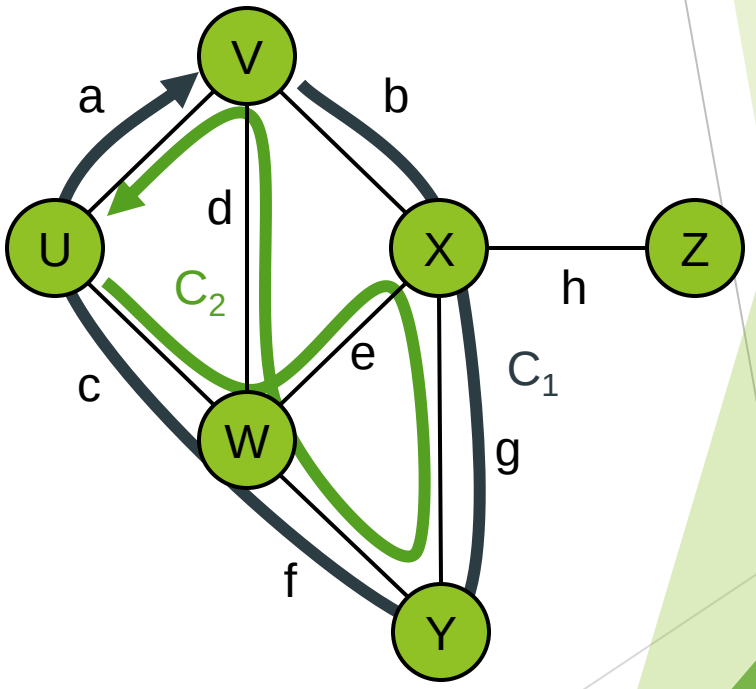
Terminology (cont'd)

- Path
  - sequence of alternating vertices and edges
  - begins with a vertex
  - ends with a vertex
  - each edge is preceded and followed by its endpoints
- Simple path
  - path such that all its vertices and edges are distinct
- Examples
  - P_1=(V,b,X,h,Z) is a simple path
  - P_2=(U,c,W,e,X,g,Y,f,W,d,V) is a path that is not simple



Terminology (cont'd)

- Cycle
  - circular sequence of alternating vertices and edges
  - each edge is preceded and followed by its endpoints
- Simple cycle
  - cycle such that all its vertices and edges are distinct (except the first and last vertex)
- Examples
  - C_1=(V,b,X,g,Y,f,W,c,U,a,↵) is a simple cycle
  - C_2=(U,c,W,e,X,g,Y,f,W,d,V,a,↵) is a cycle that is not simple

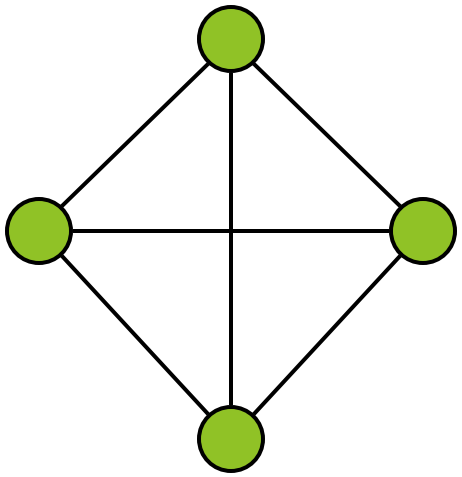


Properties

Notation

n	number of vertices
m	number of edges
$\deg(v)$	degree of vertex v

Example



- $n = 4$
- $m = 6$
- $\deg(v) = 3$

Property 1

Undirected graph: $\sum_v \deg(v) = 2m$

Proof: each edge is counted twice

Directed graph:

$\sum_v \text{indeg}(v) = \sum_v \text{outdeg}(v) = m$

Property 2

In a simple undirected graph,

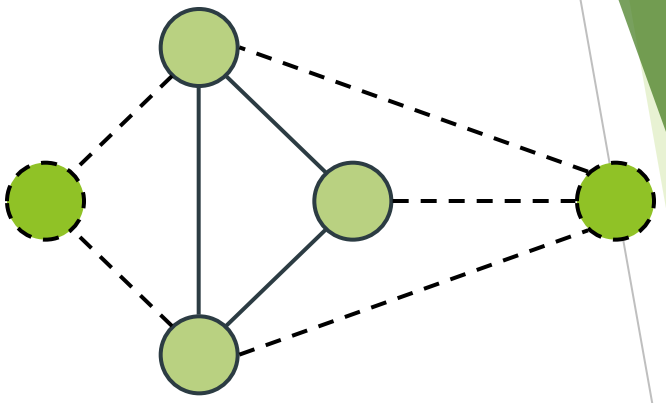
$m \leq n\,(n-1)/2$

Proof: each vertex has degree at most $(n-1)$

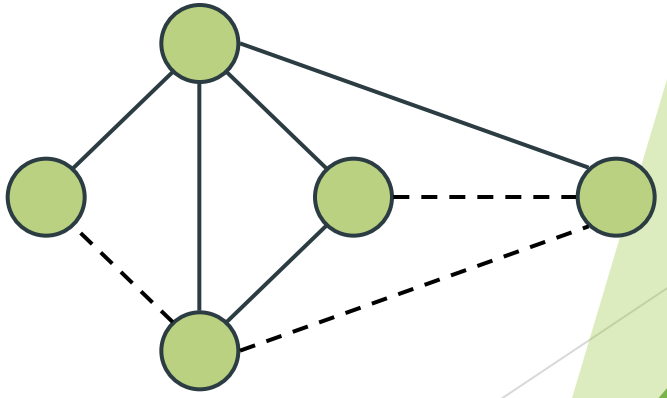
What is the bound for a directed graph?

Subgraphs

- A subgraph S of a graph G is a graph such that
  - The vertices of S are a subset of the vertices of G
  - The edges of S are a subset of the edges of G
- A spanning subgraph of G is a subgraph that contains all the vertices of G



Subgraph



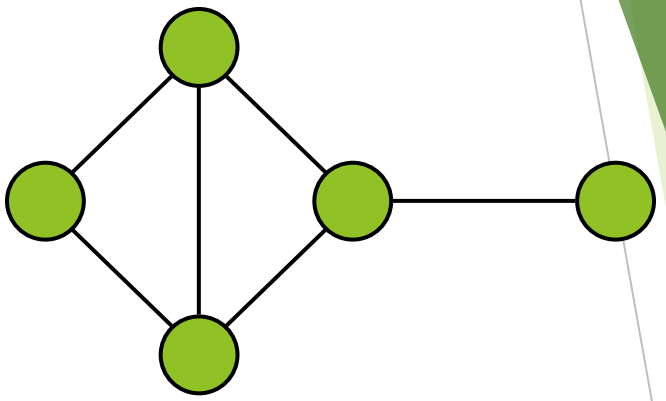
Spanning subgraph

Connectivity

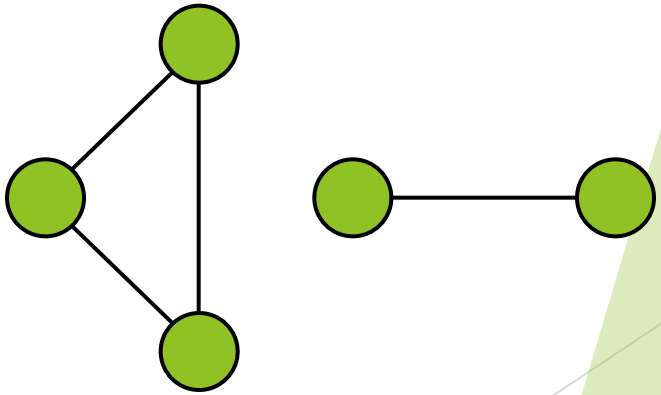
- A graph is connected if there is a path between any two vertices

- A connected component of a graph G is a maximal connected subgraph of G.

  Thus, a connected component cannot be a subgraph of another bigger connected subgraph of G



Connected graph

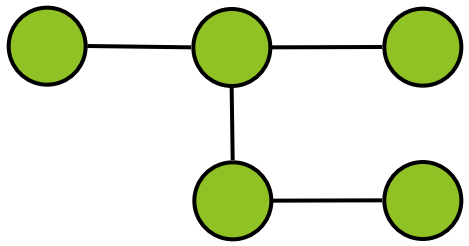


Non connected graph with two connected components

Trees and Forests

- A (free) tree is an undirected graph T such that
  - T is connected
  - T has no cycles

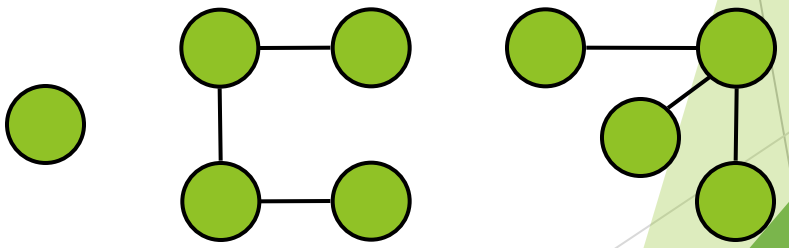
This definition of tree is different from the one of a rooted tree



Tree

- A forest is an undirected graph without cycles

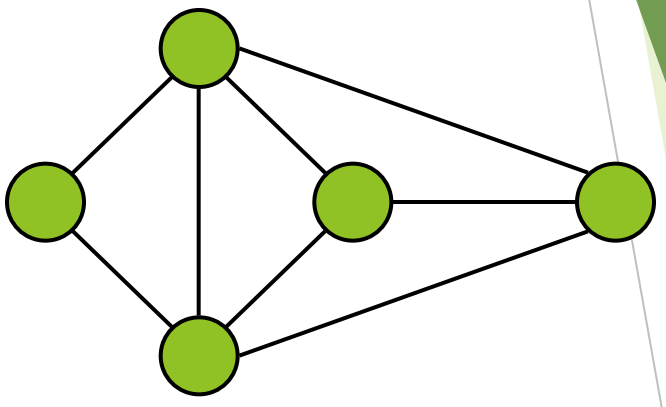
- Each connected components of a forest is a tree



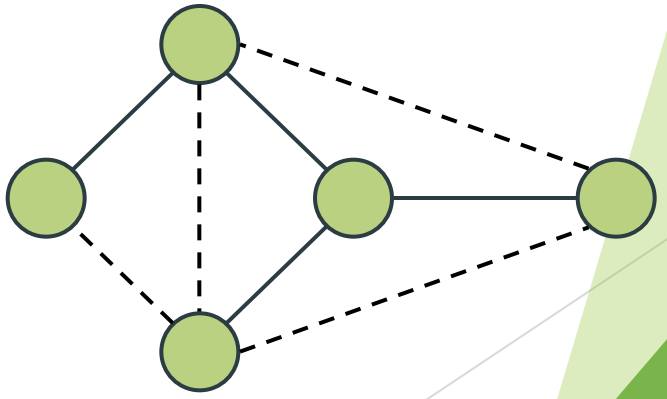
Forest

Spanning Trees and Forests

- A spanning tree of a connected graph is a spanning subgraph that is a tree
- A spanning tree is not unique unless the graph is a tree
- Spanning trees have applications to the design of communication networks
- A spanning forest of a graph is a spanning subgraph that is a forest



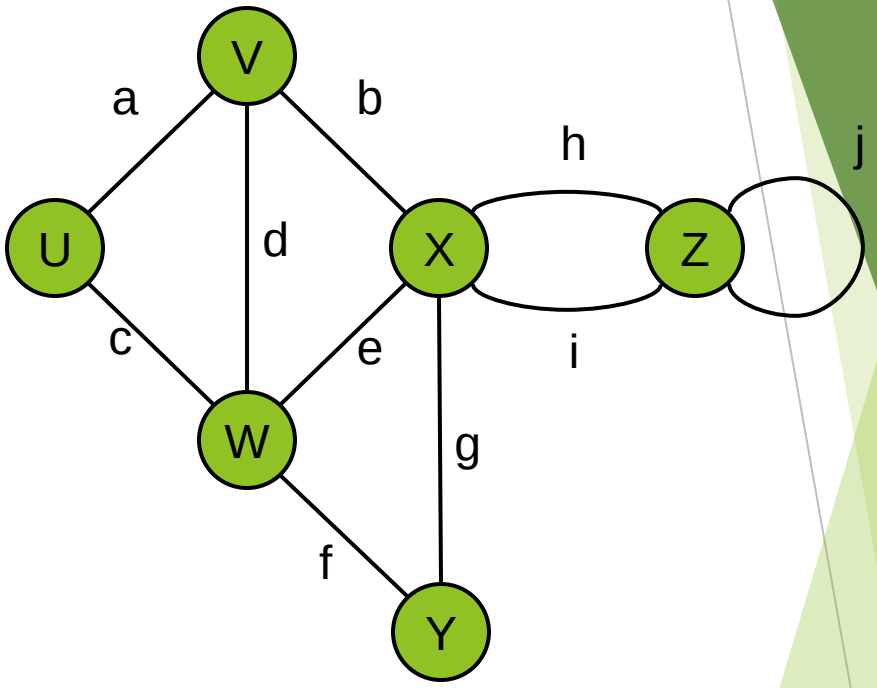
Graph



Spanning tree

Summary: Subgraphs and Trees

- Subgraph
  - Subsets of vertices and edges
  - *Spanning subgraph* (a subgraph that contains all the vertices of the original graph)
- Connected Graph
  - There is a path between any two vertices
  - *Connected components*
- Tree (free tree, no root)
  - Connected graph without circles
- Forest
  - Graph without circles
- Spanning Tree of a Graph
  - Spanning subgraph that is a (free) tree



Property: Let G be an undirected graph with n vertices and m edges, then:

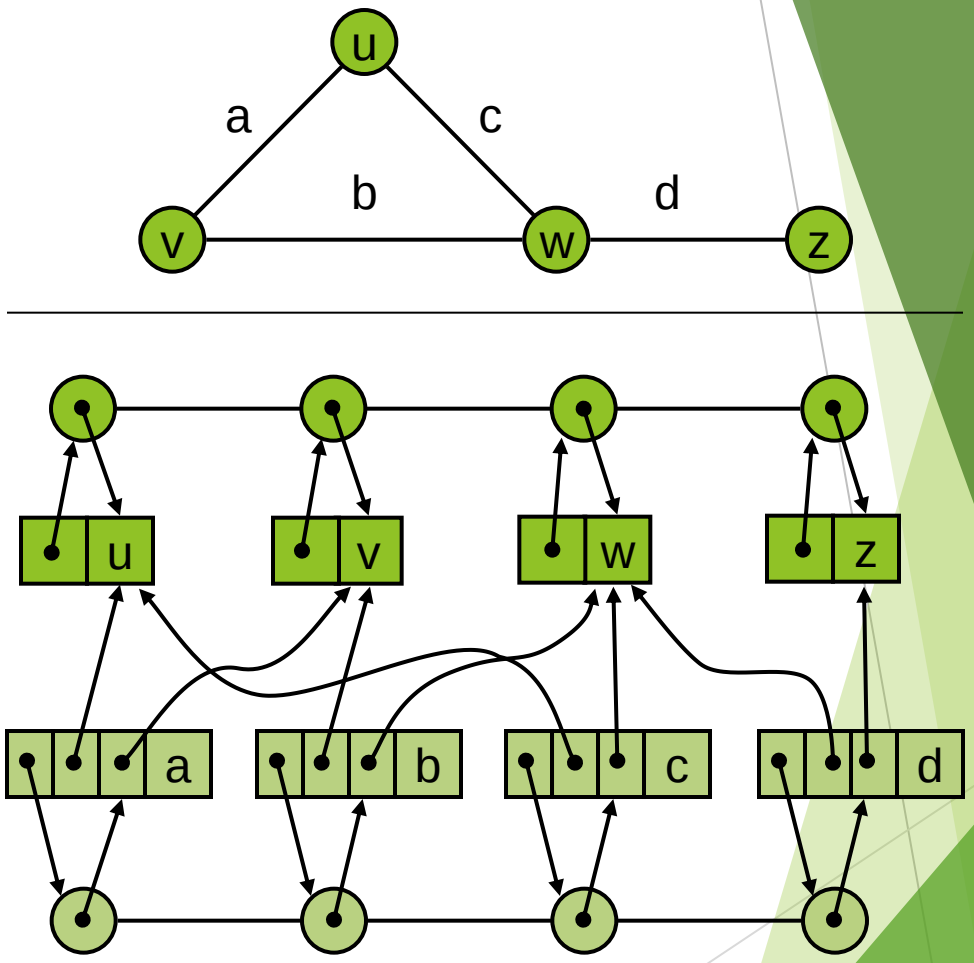
- If G is connected, then $m \geq n-1$
- If G is a tree, then $m = n-1$
- If G is a forest, then $m \leq n-1$

Main Methods of the Graph ADT

- Vertices and edges
  - are positions
  - store elements
- Accessor methods
  - aVertex(): return an arbitrary vertex of G
  - incidentEdges(v)
  - endVertices(e)
  - isDirected(e)
  - origin(e)
  - destination(e)
  - opposite(v, e)
  - areAdjacent(v, w)
- Update methods
  - insertVertex(o)
  - insertEdge(v, w, o)
  - insertDirectedEdge(v, w, o)
  - removeVertex(v)
  - removeEdge(e)
- Generic methods
  - numVertices(): # of vertices
  - numEdges(): # of edges
  - vertices(): iterator of vertices
  - edges(): iterator of edges

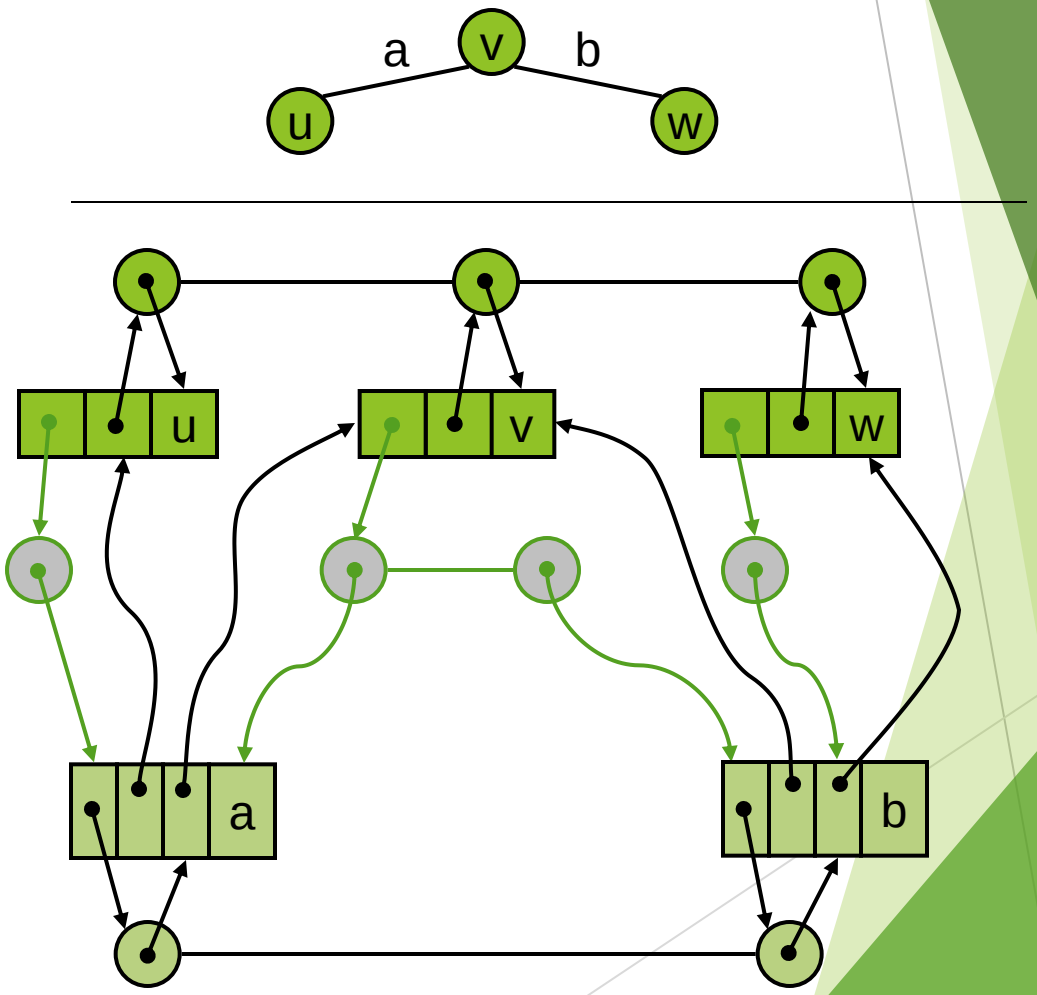
Edge List Structure

- Vertex object
  - element
  - reference to position in vertex sequence
- Edge object
  - element
  - origin vertex object
  - destination vertex object
  - reference to position in edge sequence
- Vertex sequence
  - sequence of vertex objects
- Edge sequence
  - sequence of edge objects
- Not efficient for some operations
  - e.g., IncidentEdges(v)



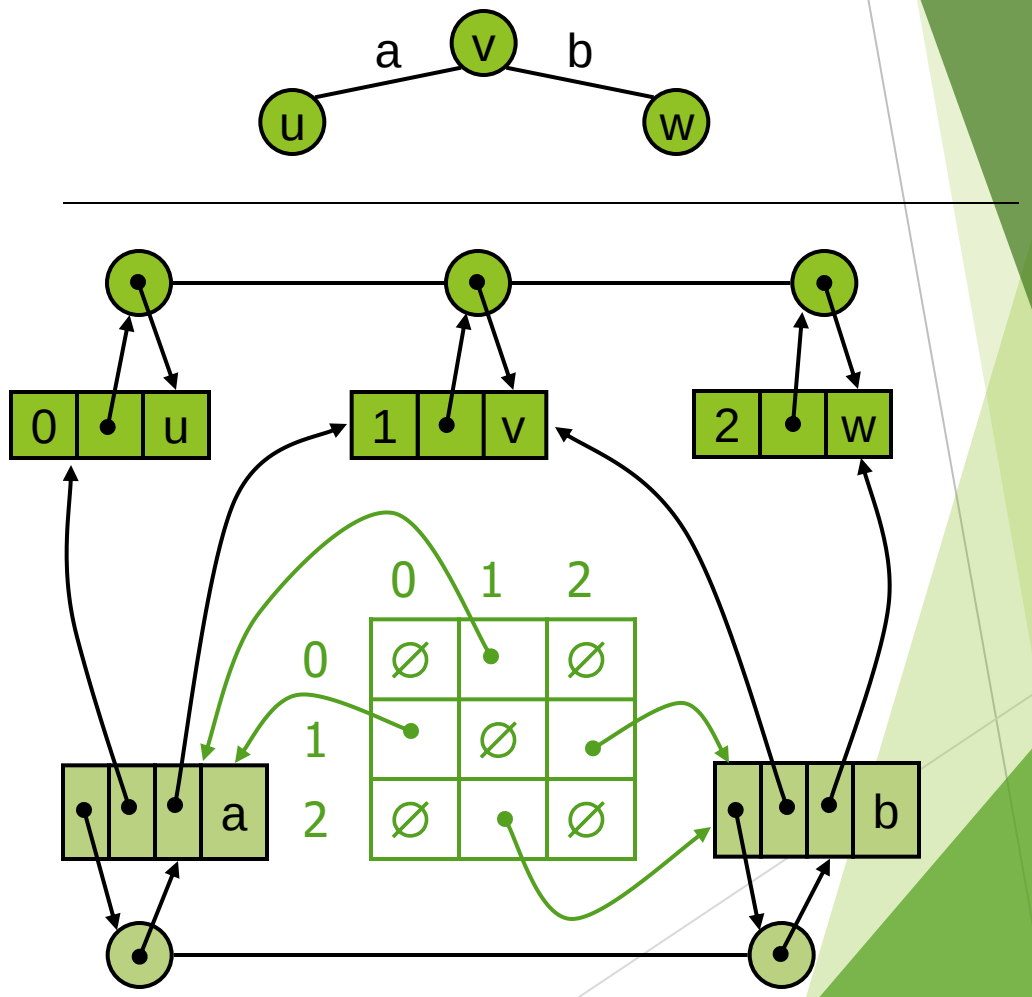
Adjacency List Structure

- Edge list structure
- Incidence sequence for each vertex
  - sequence of references to edge objects of incident edges
- Augmented edge objects
  - references to associated positions in incidence sequences of end vertices
- Good for some operations
  - IncidentEdges(v)
  - areAdjacent(u, v)
  - removeVertex(v)



Adjacency Matrix Structure

- Edge list structure
- Augmented vertex objects
  - Integer key (index) associated with vertex
- 2D adjacency array
  - Reference to edge object for adjacent vertices
  - Null for non nonadjacent vertices
  - May be simplified (store edge info in the matix)
  - Symmetric (undirected) or non-symmetric (directed)
- The “old fashioned” version just has 0 for no edge and 1 for edge



Asymptotic Performance

◆ n vertices, m edges ◆ no parallel edges ◆ no self-loops ◆ Bounds are "big-Oh"	Edge List	Adjacency List	Adjacency Matrix
Space	$n + m$	$n + m$	n^2
incidentEdges(v)	m	$\deg(v)$	n
areAdjacent (v, w)	m	$\min(\deg(v), \deg(w))$	1
insertVertex(o)	1	1	n
insertEdge(v, w, o)	1	1	1
removeVertex(v)	m	$\deg(v)$	n^2
removeEdge(e)	1	1	1